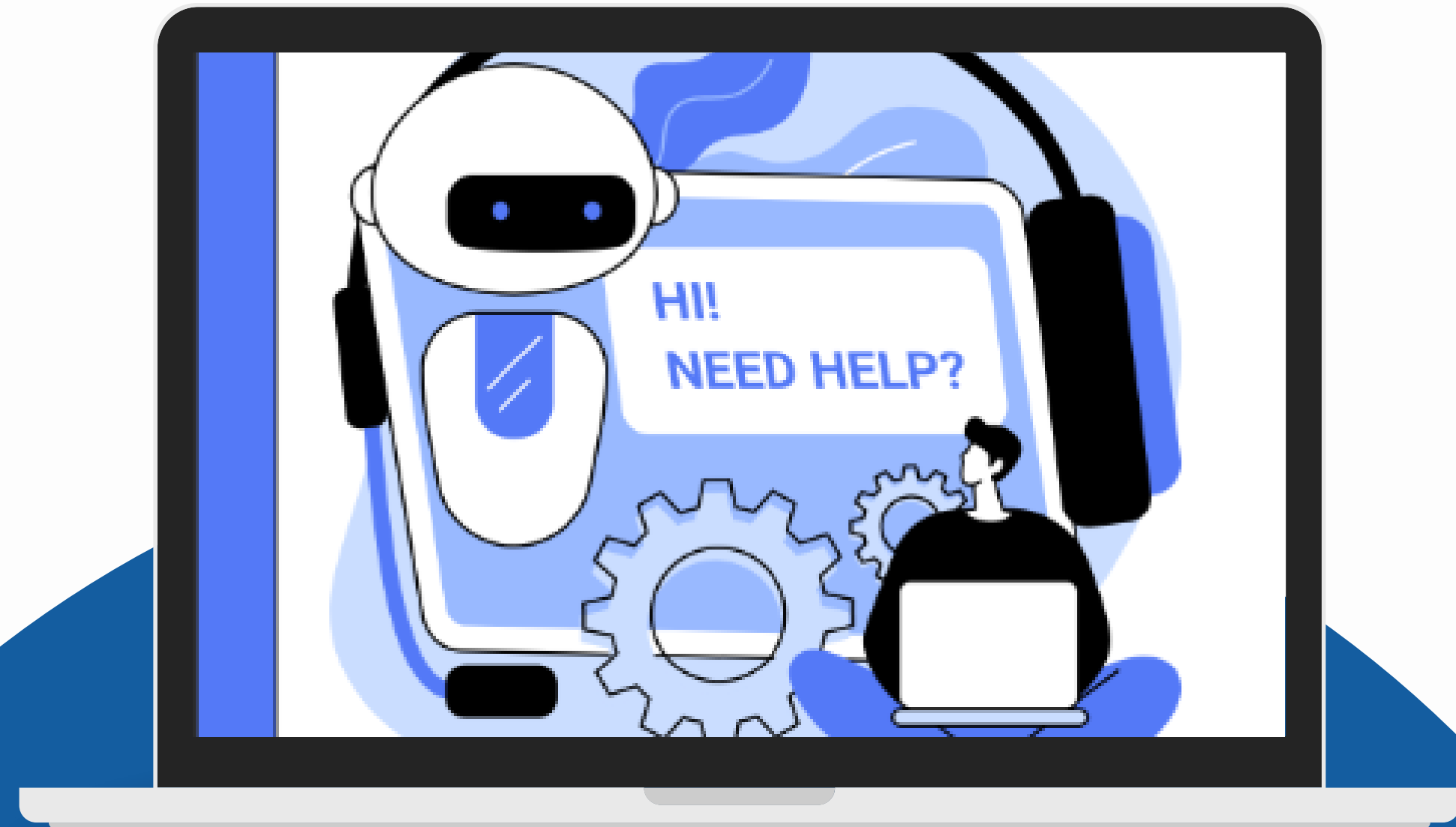


Segmentation and Paging in Memory Management

Presented by: Nandita Singh
Regd no.-2141019333



Overview

- ▶ Introduction
- ▶ What is memory management
- ▶ why memory management matters
- ▶ What is paging
- ▶ what is segmentation
- ▶ Paging vs Segmentation
- ▶ Real life analogy
- ▶ Summary



What Is
Memory
Management ?



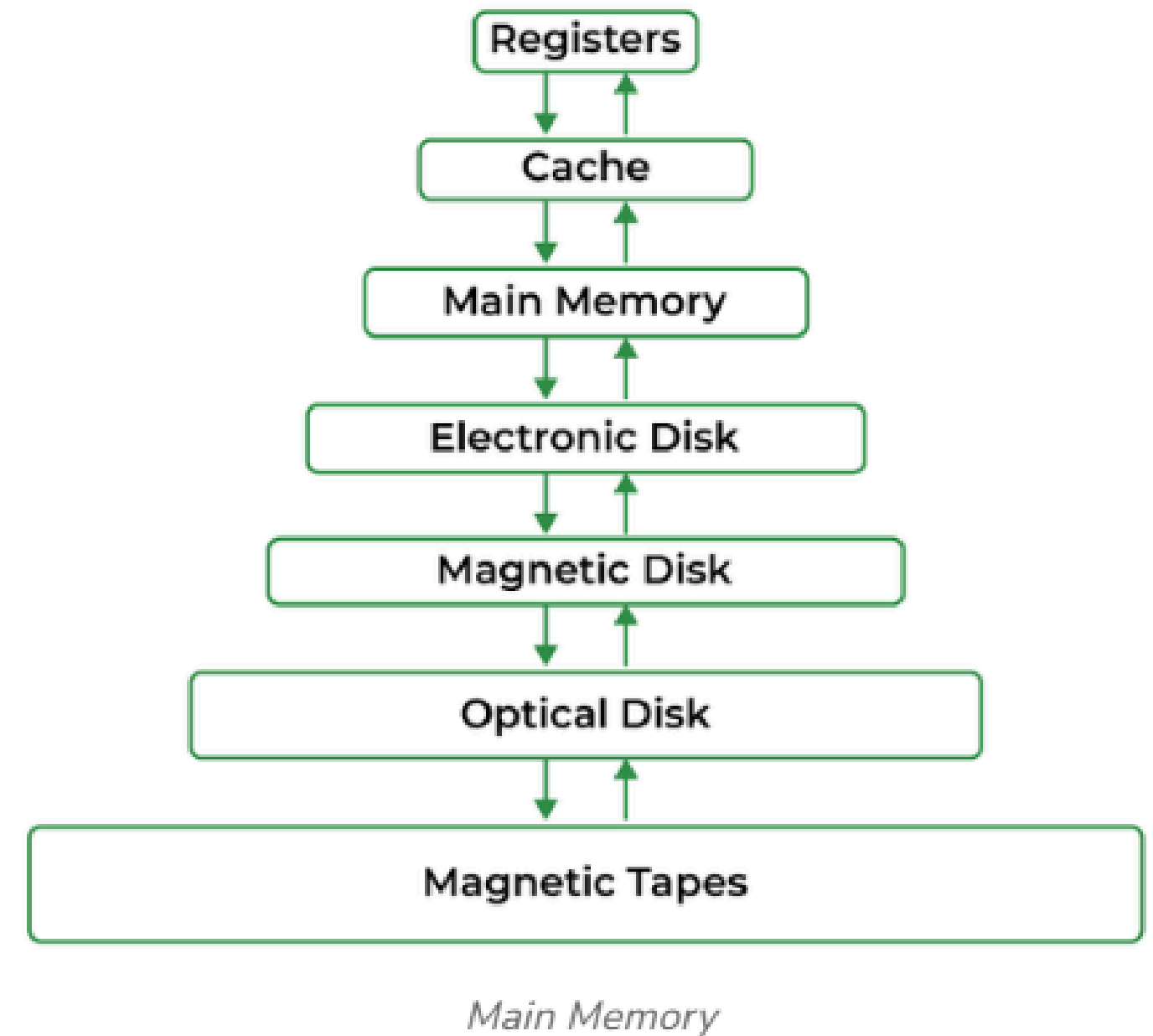
Introduction

- Imagine trying to store different types of files—music, videos, documents—all in a single long list on your computer. It would be slow, disorganized, and hard to manage. That's how memory would behave without structure.
- Segmentation and paging are two core techniques that organize memory efficiently, just like organizing files into folders or arranging books on a shelf. They help the operating system allocate memory wisely, protect processes from each other, and support fast, multitasking environments.
- This presentation explores how these methods work, why they matter, and how they are used in real-world systems like the Intel x86 architecture.

What is memory Managment ??

Memory management is a core function of an operating system that handles or manages primary memory. It is responsible for:

- Keeping track of each byte in a computer's memory.
- Allocating memory to processes when they need it.
- Deallocating memory when it is no longer in use.
- Ensuring processes do not interfere with each other's memory (protection).
- Optimizing the performance of the system by minimizing fragmentation.



Why Memory Management matters ??

01

Poor memory allocation can lead to system crashes, application errors, and security issues. Proper memory management enables:

02

Multitasking: Run multiple programs simultaneously

03

Isolation: Prevent memory corruption across processes

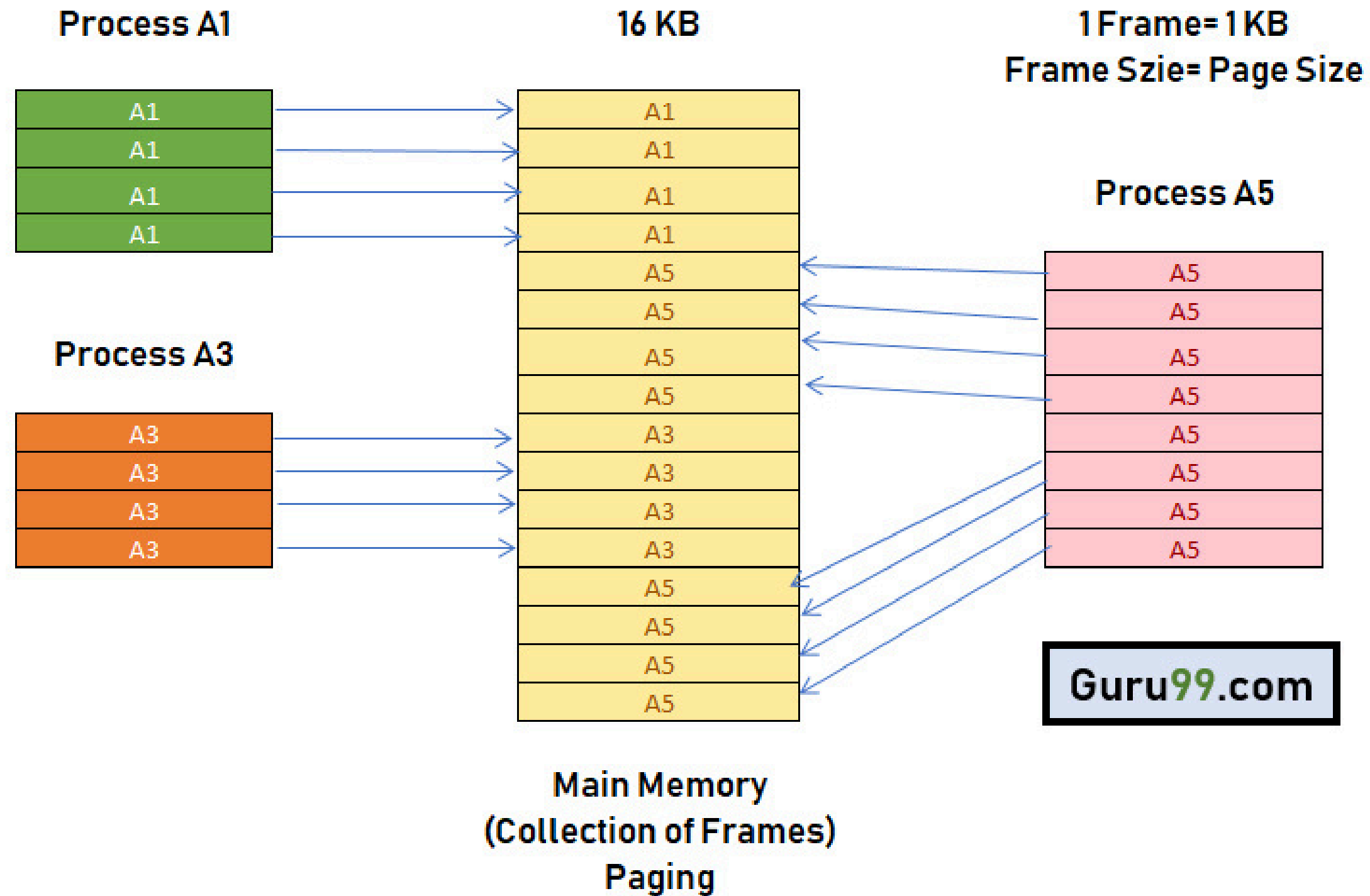
04

Efficiency: Maximize use of RAM and reduce fragmentation

What is Paging??

Definition: Paging is a memory management scheme that divides logical memory into fixed-size blocks called pages and physical memory into blocks called frames. The OS uses a Page Table to map logical pages to physical frames.

- Memory is split into equal-sized pages (logical) and frames (physical).
- Each logical page maps to any available physical frame.
- Uses a page table for mapping.
- Solves external fragmentation, but may cause internal waste. Visual: Show logical address → page table → physical memory.



Paging Address Translation Example

- Page size = 1024 Bytes (1 KB)
- Logical Address = 2049
- Page Number = $2049 / 1024 = 2$
- Offset = $2049 \% 1024 = 1$
- Assume Page 2 → Frame 5 → Physical Address = $5 \times 1024 + 1 = 5121$

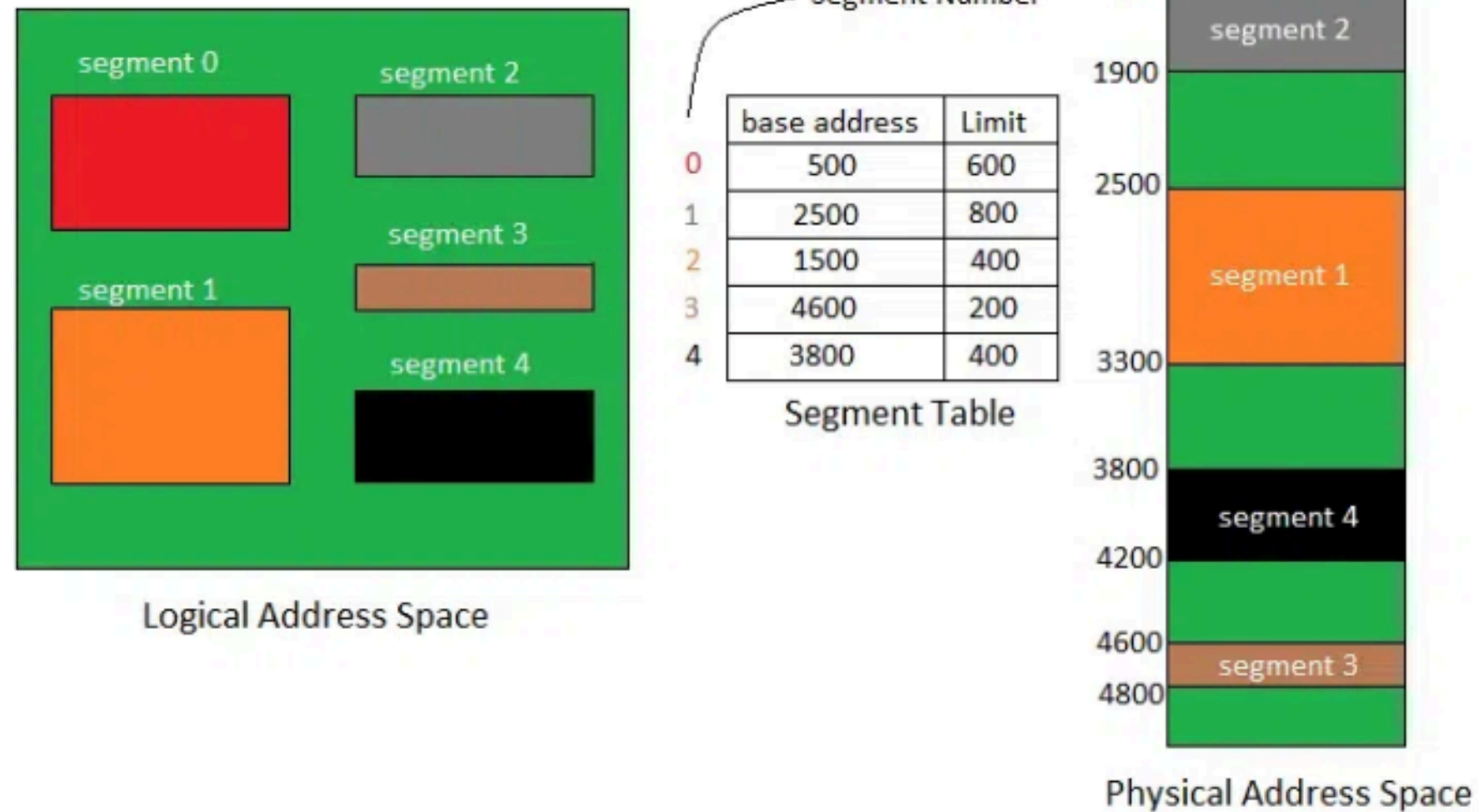
Guru99.com

What is Segmentation ??

Definition: Segmentation divides memory based on logical program segments such as code, data, and stack. Each segment is independently sized and managed.

- Memory divided into logical segments (e.g., code, data, stack).
- Each segment has a base (start address) and limit (size).
- Better matches programmer's view of memory. Visual: Program broken into labeled blocks: Code, Data, Stack, etc.

Logical View of Segmentation



Segmentation

Segmentation Address Translation Example

- Segment 2: Base = 1000, Limit = 500
- Logical Address = (2, 120)
Translation:
- Check: Offset (120) < Limit (500) → Valid
- Physical Address = Base + Offset = 1000 + 120 = 1120

Paging vs Segmentation

Paging

- Divides memory into fixed-size blocks called pages (logical memory) and frames (physical memory).
- Uses a page number and an offset to address memory. The page table is used to map logical pages to physical frames.
- Allocates memory in fixed-size blocks, avoiding external fragmentation but can cause internal fragmentation if a page is not fully utilized.

Segmentation

- Divides memory based on logical units like code, stack, and data. Each segment can have a different size based on the program's structure.
- Uses a segment number and an offset. Each segment has a base address and limit (size).
- Allocates memory based on the logical structure of a program, but it can cause external fragmentation over time as segments vary in size.

REAL-LIFE ANALOGY

- Paging: Like dividing a large book into equal-sized chapters, stored in slots on a shelf. Easy to manage, but may waste space (internal fragmentation).
- Segmentation: Like storing different types of books (novels, dictionaries, magazines) in separate sections. Matches real-world use but requires varied shelf sizes (external fragmentation).
- Combined: Like storing books by category (segment) and then by size (page).

Summary

- Paging is a memory management method that breaks both logical and physical memory into equal-sized blocks (pages and frames). It simplifies memory allocation and avoids external fragmentation but can lead to internal fragmentation. The use of a page table ensures efficient address translation.
- Segmentation divides memory into variable-sized logical segments based on the structure of a program (like code, stack, and data). It allows for more intuitive and flexible memory use, offering better logical separation and protection but may suffer from external fragmentation.
- Both techniques aim to solve memory allocation challenges in their own ways. Paging emphasizes uniformity and efficiency, while segmentation provides better logical organization and flexibility.
- In modern systems, especially in architectures like x86, both segmentation and paging are combined. Segmentation provides logical grouping, and paging manages memory within those groups, allowing for optimized memory use, virtual memory support, and enhanced protection.

THANK YOU!